EventAdda — Implementation Plan

Hyperlocal Event Discovery & Community Engagement Platform

Prepared for	VecnaVerse National Level Hackathon
Domain	Software Development
Version	1.0
Date	October 2026
Stack	React · FastAPI · MongoDB

1. Executive Summary

Events such as hackathons, workshops, summits, conferences, and meetups are scattered across WhatsApp groups, Instagram stories, LinkedIn posts, physical posters, and dozens of independent websites. Participants waste time manually searching multiple platforms and still miss relevant events in their locality. Organizers, in turn, struggle to reach the right audience efficiently.

EventAdda solves this by centralizing event discovery into a single location-aware platform. Participants complete a lightweight onboarding — selecting interests and a preferred city — and immediately receive personalized event recommendations. They can browse events by four primary categories (Hackathons, Workshops, Summits & Conferences, Meetups), view detailed event information, and register directly within the platform. Organizers create, publish, and manage events while viewing their registered participants. In-app and email notifications keep everyone informed of important changes.

The technical architecture is a **modular monolith**: a React single-page application communicating via REST/JSON with a FastAPI backend, backed by MongoDB. This approach balances clean separation of concerns with the practical simplicity required for a hackathon timeline. The recommendation engine uses a lightweight, explainable scoring algorithm based on interest matching, location proximity, and event freshness — no machine-learning infrastructure required.

This document provides the complete implementation guide: confirmed requirements, unresolved decisions with recommendations, MongoDB data models, a full API contract, authentication design,

the recommendation algorithm, notification architecture, a dependency-aware phased implementation plan, testing strategy, and team coordination workflow.

2. Problem Statement and Product Vision

2.1 The Problem

The event ecosystem in India is deeply fragmented:

- **Hackathons** are announced on Devfolio, Unstop, and college WhatsApp groups
- **Workshops** appear on Instagram stories, Meetup.com, and email newsletters
- **Summits and conferences** are promoted on LinkedIn, Twitter, and organizer websites
- **Meetups** are shared in Telegram channels and local community groups

A student in Pune looking for upcoming hackathons must check five or more platforms daily. An organizer hosting a workshop in Bangalore has no single channel to reach interested participants in that city. The result: participants miss events they would have loved, and organizers fail to fill seats with the right audience.

2.2 The Solution

EventAdda is a **hyperlocal event discovery platform** with two core user journeys:

1. **Participants** create an account, select their interests and city, and immediately see a personalized dashboard of relevant nearby events. They click on category cards (Hackathons, Workshops, etc.) to drill into filtered listings, view event details, and register — all within EventAdda.
2. **Organizers** create event listings (as drafts or published), manage event details, and view their registered participants with contact information.

2.3 Core Value Proposition

For	Pain Point	EventAdda Solution
Participants	Must search 5+ platforms to find events	One dashboard with personalized recommendations
Participants	Miss nearby events they'd enjoy	Location-aware discovery + interest matching
Participants	Redirected to external registration pages	In-app registration with confirmation
Organizers	Struggle to reach the right audience	Events surfaced to interested, nearby participants
Organizers	No centralized view of registrations	Dashboard with registered participant details

3. Workspace Inspection Results

Check	Status
Git repository	❌ Not initialized
Frontend files / Stitch exports	❌ None found
Backend code	❌ None found
Package manifests	❌ None found
Configuration files	❌ None found
Documentation	❌ None found

Conclusion: The workspace at `D:\Coding Projects\@Projects\EventAdda` is completely empty. There is no existing code, configuration, or documentation to preserve. The implementation will start from scratch.

4. Confirmed Requirements Summary

4.1 Branding & Welcome Screen

Requirement	Details
App name	EventAdda
Welcome screen style	Minimalist, uncluttered, visually polished
Entry points	Two clear buttons: "Participant" and "Organizer"
Role selection behavior	Leads to authentication flow; does NOT grant authorization
Backend enforcement	Account permissions enforced server-side regardless of UI role selection

4.2 Participant Onboarding

Field	Required	Collected At
Full name	✅	Signup
Email	✅	Signup
Phone number	✅	Signup
Password	✅	Signup
Interests (≥1)	✅	Onboarding (post-signup)
Preferred location	✅	Onboarding (post-signup)
Browser geolocation	❌ Optional	Onboarding

> **IMPORTANT**
>
> Keep personal data collection minimal. No mandatory college, age, gender, academic year, or biography fields.

4.3 Participant Dashboard

The dashboard contains:

- **Personalized recommendations** near the top

- **Four category cards**: Hackathons · Workshops · Summits & Conferences · Meetups
- Event search and discovery
- Location information and location-based filtering
- Access to registered events ("My Events")
- Notifications
- Profile and preference management

Category card behavior: Clicking a category card (e.g., "Hackathons") navigates to a filtered listing showing all published events in that category. Each event listing displays the event card with title, date, venue/city, registration status, and capacity information. The user can click into any event for full details and registration.

4.4 Location-Aware Discovery

Capability	Details
Browser geolocation	Requested with permission; fully optional
Manual selection	User picks city/locality from a list
Event venues	Each event has venue information with city
Location filtering	Filter or rank events by proximity to user's location
Location recommendations	Nearby events scored higher in recommendations
Access control	Location does NOT restrict access — users can view events anywhere

4.5 Registration

Rule	Details
Registration location	In-app only; no external links required
Duplicate prevention	One registration per participant per event
Capacity enforcement	Registration blocked when `registered_count >= max_capacity`
Deadline enforcement	Registration blocked after `registration_deadline`
Cancellation	Participant can cancel until event starts
Capacity release	Cancelled registration releases one seat
Confirmation	Registration confirmation notification
Concurrency	Atomic operations prevent race conditions

4.6 Organizer Workflow

Capability	Details
Create events	Draft or publish immediately
Edit published events	Changes reflected for participants
Cancel events	Triggers notifications to registrants
View registrations	Name, email, phone, registration date
View counts	Registration count per event
Authorization	Organizer can only manage own events
Verification/approval	OUT OF SCOPE for hackathon

4.7 Notifications

Trigger	Channel	Details
Registration confirmed	In-app + Email	Participant registers successfully
Registration cancelled	In-app	Participant cancels registration
Event date/time/venue changed	In-app + Email	Important change to registered event
Event cancelled	In-app + Email	Registered event cancelled by organizer
Recommendations	Dashboard only	NOT sent as notifications

4.8 Event Data

- Use **dummy event entries** for development and demos
- External scraping / third-party ingestion: NOT required
- Organizer-created events must be fully supported
- Provide a seed script for demo data + safe reset

4.9 UI/UX Direction

Aspect	Direction
Style	Modern minimalist, restrained colors
Layout	Persistent left sidebar on desktop
Priority	Desktop-first for hackathon demo
Responsiveness	Sensible for smaller screens
Events	Clear event cards, readable details
Recommendations	Prominently displayed
Onboarding	Minimal friction
Experiences	Separate participant and organizer flows

5. Unresolved Decisions and Recommendations

5.1 Account Roles

> **Status: ⚠ PENDING TEAM CONFIRMATION**

The team has not decided whether each account has exactly one role or can act as both participant and organizer.

Option	Description	Pros	Cons
A: Single role	Account is either "participant" or "organizer"	Simpler auth logic; clear separation	Users need two accounts to both organize and attend; poor UX
B: Dual role ☆	Account has a `role` field: `"participant"`, `"organizer"`, or `"both"`	One login; natural for hackathon users who organize and attend; flexible	Slightly more complex role checks

Recommendation: Option B (Dual Role)

Rationale: In the hackathon community, many users both organize and attend events. Forcing separate accounts creates friction and data duplication. A `role` field with three values keeps the auth model simple while providing a natural user experience.

Implementation impact:

- User model: `role` field with enum `["participant", "organizer", "both"]`
- Role checking: `require_role(["participant", "both"])` for participant actions, `require_role(["organizer", "both"])` for organizer actions
- Welcome screen: role selection determines initial signup role; users can upgrade to "both" later
- Dashboard: role-switcher UI element for users with role "both"

5.2 Verification Strategy

> **Status: ⚠ PENDING TEAM CONFIRMATION**

Phase	What	When	Effort
MVP (Hackathon)	Collect email + phone, enforce uniqueness at DB level, `email_verified: false`	Signup	Low
Post-MVP	Email verification via tokenized link (console-logged in dev)	After signup	Medium
Future	Phone OTP via MSG91 or Twilio	After signup	High

Recommendation for hackathon: Collect and validate format, enforce uniqueness, skip actual verification. Mark fields as `email_verified: false` and `phone_verified: false` so the schema is ready for verification later. The application works without verification — it's a security improvement, not a blocker.

Key distinction:

- ✅ **Collecting** email/phone = required (signup fields)
- ✅ **Uniqueness validation** = required (MongoDB unique indexes)
- ⚠️ **Identity verification** = deferred (not blocking hackathon)

5.3 Other Open Questions

#	Question	Why It Matters	Options	Recommended Default	Consequence of Postponing
1	**Event image uploads**	Visual appeal of event cards	A) URL-based (paste link) B) File upload to server C) Cloud storage (S3/Cloudinary)	**A: URL-based** — organizer pastes an image URL; simplest for MVP	Events without images need a placeholder; file upload adds complexity
2	**Search implementation**	Discovery experience	A) MongoDB text index B) Regex search C) Elasticsearch	**A: MongoDB text index** — built-in, no extra infrastructure	Text search quality is basic but functional
3	**Payment / ticketing**	Monetization	A) Free events only B) Integrate Razorpay	**A: Free only** — `is_free: true` for all MVP events	Out of scope; field exists for future
4	**Event analytics**	Organizer value	A) Basic registration count B) Detailed analytics dashboard	**A: Basic counts** — `registered_count` on event document	Analytics deferred post-MVP
5	**Data retention**	Storage and privacy	A) Keep everything B) TTL indexes on notifications C) Manual cleanup	**B: TTL index** on notifications (90 days); events and registrations retained	Old notifications accumulate without TTL
6	**Password reset**	Account recovery	A) Implement reset via email B) Defer	**B: Defer** — no password reset for MVP; can be added with email verification	Users who forget passwords must create new accounts

6. User Journeys and Workflows

Workflow 1: Participant Account Creation

Aspect	Details
Purpose	Create a new participant account
Actors	Unregistered user
Prerequisites	None
Steps	1. Select "Participant" on welcome screen → 2. Fill signup form (name, email, phone, password) → 3. Submit → 4. Account created, JWT returned
Success	Account exists in DB with `role: "participant"`, `onboarding_completed: false`; user receives JWT
Failure cases	Email already exists (409) · Phone already exists (409) · Invalid email format (422) · Weak password (422) · Missing required fields (422)

Workflow 2: Participant Onboarding

Aspect	Details
Purpose	Collect interests and location to enable personalized recommendations
Actors	Authenticated participant with `onboarding_completed: false`
Prerequisites	Completed signup; valid JWT
Steps	1. Select ≥1 interest from predefined list → 2. Set preferred location (city, optional locality) → 3. Optionally grant browser geolocation → 4. Complete onboarding
Success	User document updated with interests array and location; `onboarding_completed: true`
Failure cases	Zero interests selected (400) · No city provided (400) · Token expired (401)

Workflow 3: Location Selection

Aspect	Details
Purpose	Establish the user's preferred location for discovery
Actors	Participant during onboarding or profile edit
Prerequisites	Authenticated
Steps	1. Frontend requests browser geolocation permission → 2a. If granted: coordinates sent to backend, city resolved from static lookup → 2b. If denied: user selects city from dropdown list → 3. Location saved to user profile
Success	User has `location: { city, locality?, coordinates? }`
Failure cases	Geolocation denied + no manual selection (400) · Invalid city name (422)

Workflow 4: Personalized Dashboard

Aspect	Details
Purpose	Show relevant events tailored to the participant
Actors	Authenticated participant with completed onboarding
Prerequisites	Interests and location set
Steps	1. Load dashboard → 2. Backend scores eligible events → 3. Top recommendations displayed near top → 4. Four category cards shown → 5. Location info displayed
Success	Dashboard renders with recommendations, category cards, location, and navigation
Failure cases	No eligible events (show "No events found" with suggestion) · Onboarding incomplete (redirect to onboarding)

Workflow 5: Event Browsing and Filtering

Aspect	Details
Purpose	Discover events by category, city, date, or keyword
Actors	Any user (public browsing) or authenticated participant
Prerequisites	None for public; auth for personalized results
Steps	1. Browse all events or apply filters → 2. Filter by category, city, date range, search query → 3. View paginated results as event cards
Success	Filtered event list displayed with relevant cards
Failure cases	No matching events (empty results with message)

Workflow 6: Category Card Drill-Down

Aspect	Details
Purpose	View all events in a specific category when clicking a category card
Actors	Any user
Prerequisites	None
Steps	1. Click category card (e.g., "Hackathons") on dashboard → 2. Navigate to category-filtered view → 3. See all published events in that category with event cards showing title, date, city, capacity status → 4. Click any event card for full details
Success	Category-filtered event listing displayed with event details
Failure cases	No events in category (show empty state)

Workflow 7: Event Details

Aspect	Details
Purpose	View complete information about a specific event
Actors	Any user
Prerequisites	Event exists and is published (or user is the organizer for drafts)
Steps	1. Click event card → 2. View full details (title, description, date/time, venue, category, tags, capacity, registration status) → 3. See "Register" button if eligible
Success	Full event details rendered; registration eligibility shown
Failure cases	Event not found (404) · Event cancelled (shown with "cancelled" badge)

Workflow 8: Event Registration

Aspect	Details
Purpose	Register for an event
Actors	Authenticated participant
Prerequisites	Event published; deadline not passed; capacity available; not already registered
Steps	1. Click "Register" → 2. Backend atomically checks capacity and creates registration → 3. `registered_count` incremented → 4. Confirmation notification created → 5. Confirmation email attempted
Success	Registration document created with `status: "confirmed"`; notification sent
Failure cases	Already registered (409) · Capacity full (400) · Deadline passed (400) · Event cancelled (400) · Not authenticated (401)

Workflow 9: Registration Cancellation

Aspect	Details
Purpose	Cancel a registration before the event starts
Actors	Authenticated participant who is registered
Prerequisites	Active registration exists; event hasn't started
Steps	1. Go to My Events → 2. Click "Cancel Registration" → 3. Registration status set to "cancelled" → 4. `registered_count` decremented → 5. Cancellation notification created
Success	Registration `status: "cancelled"`; capacity released
Failure cases	No active registration (404) · Event already started (400) · Event already cancelled (400)

Workflow 10: My Events

Aspect	Details
Purpose	View events the participant is registered for
Actors	Authenticated participant
Prerequisites	Auth required
Steps	1. Navigate to My Events → 2. See list of registered events (upcoming and past) → 3. Each shows event title, date, venue, registration status
Success	List of registered events displayed; upcoming events allow cancellation
Failure cases	No registrations (show empty state)

Workflow 11: Organizer Event Creation

Aspect	Details
Purpose	Create a new event
Actors	Authenticated organizer
Prerequisites	User has role "organizer" or "both"
Steps	1. Fill event form (title, description, category, venue, dates, capacity, etc.) → 2. Choose "Save as Draft" or "Publish" → 3. Event created with appropriate status
Success	Event document created with `organizer_id` set to current user
Failure cases	Missing required fields (422) · Invalid dates (422) · Not an organizer (403)

Workflow 12: Draft Saving and Publishing

Aspect	Details
Purpose	Save event as draft and publish when ready
Actors	Organizer (event owner)
Prerequisites	Event exists in draft status; user owns the event
Steps	1. Create event with `status: "draft"` → 2. Edit as needed → 3. Publish via PATCH endpoint → 4. Status changes to "published" → 5. Event appears in public listings
Success	Event visible to participants after publishing
Failure cases	Not the event owner (403) · Already published (400) · Already cancelled (400)

Workflow 13: Published Event Editing

Aspect	Details
Purpose	Update a published event's details
Actors	Organizer (event owner)
Prerequisites	Event is published; user owns the event
Steps	1. Edit event details → 2. Submit changes → 3. If date/time/venue changed, create notifications for all registrants → 4. Updated event reflected for participants
Success	Event updated; registrants notified of important changes
Failure cases	Not owner (403) · Event cancelled (400)

Workflow 14: Organizer Registration Management

Aspect	Details
Purpose	View registered participants for an event
Actors	Organizer (event owner)
Prerequisites	Event exists; user owns it
Steps	1. Open event management → 2. View registrations list → 3. See participant name, email, phone, registration date
Success	Paginated list of registrants displayed
Failure cases	Not owner (403) · Event not found (404)

Workflow 15: Event Cancellation

Aspect	Details
Purpose	Cancel a published event
Actors	Organizer (event owner)
Prerequisites	Event is published or draft; user owns it
Steps	1. Click "Cancel Event" → 2. Event status set to "cancelled" → 3. Notifications created for all registrants → 4. Email notifications attempted → 5. Event no longer appears in public listings
Success	Event cancelled; all registrants notified
Failure cases	Not owner (403) · Already cancelled (400)

Workflow 16: In-App Notifications

Aspect	Details
Purpose	Inform users of important events
Actors	System → User
Prerequisites	Triggering action occurs (registration, event change, cancellation)
Steps	1. Trigger event occurs → 2. Notification document created → 3. User sees notification indicator → 4. User opens notifications → 5. Marks as read
Success	Notification visible, unread count accurate, mark-as-read works
Failure cases	Notification creation fails (log error, don't crash the trigger action)

Workflow 17: Email Notifications

Aspect	Details
Purpose	Deliver important updates via email
Actors	System → User's email
Prerequisites	User has email; trigger is email-worthy
Steps	1. In-app notification created first → 2. Email attempt made (fire-and-forget) → 3. Success: email delivered → 3alt. Failure: logged, in-app notification still exists
Success	Email delivered; in-app notification exists regardless
Failure cases	SMTP failure (logged, not blocking) · Invalid email (logged)

Workflow 18: Profile and Interest Management

Aspect	Details
Purpose	Update profile, interests, or location after onboarding
Actors	Authenticated user
Prerequisites	Auth required
Steps	1. Open profile settings → 2. Edit name, phone, interests, location → 3. Save changes
Success	Profile updated; future recommendations use new preferences
Failure cases	Phone number already taken (409) · Empty interests (422)

7. Technical Feasibility Review

7.1 Critical Path Dependencies

```
Phase 0: Workspace Setup
    ↓
Phase 1: Backend Foundation (FastAPI + MongoDB)
    ↓
Phase 2: Authentication ← blocks ALL protected features
    ↓
Phase 3: Events + Seed Data ← blocks discovery, registration
    ↓
Phase 4: Onboarding + Recommendations ← blocks personalized dashboard
    ↓
Phase 5: Registrations ← blocks My Events, organizer management
    ↓
Phase 6: Notifications ← depends on registrations + event updates
    ↓
Phase 7: Frontend Integration
    ↓
Phase 8: Testing + Demo
```

Authentication (Phase 2) is the single most critical blocker. Everything beyond public event browsing depends on it.

7.2 High-Risk Implementation Areas

Area	Risk	Mitigation
Capacity enforcement	Race conditions under concurrent registrations	MongoDB `findOneAndUpdate` with `$inc` and conditions
Recommendation engine	Complex scoring may be slow or produce poor results	Simple weighted scoring; test with seed data; cache 15 min
Email delivery	External SMTP may be unavailable or unreliable	Console fallback as default; email is fire-and-forget
Geospatial queries	2dsphere indexes and `$geoNear` can be tricky	Fallback to city string matching; coordinates optional
Frontend integration	Schema mismatches between React and FastAPI	Shared API contract document; mock response files

7.3 Features That Must Work for Demo

1. ✅ Signup and login with JWT
2. ✅ Onboarding (interests + city selection)
3. ✅ Dashboard with recommendations
4. ✅ Category cards that drill down to filtered listings
5. ✅ Event details page
6. ✅ Event registration with capacity check
7. ✅ My Events
8. ✅ Organizer event creation and publishing
9. ✅ Organizer view registrations
10. ✅ In-app notification on registration

7.4 Features That Can Be Simplified

Feature	Simplification
Email notifications	Console logging instead of real SMTP
Location scoring	City-name matching instead of distance calculation
Search	MongoDB text index instead of full-text search engine
Password reset	Defer entirely
Rate limiting	Defer or add only to login
Event images	URL field with placeholder fallback